

Chapitre 7

Algorithmes de Jointure

Soit les relations suivantes :

- **Directeur**(*nom_directeur*, *nom_film*)
- **Acteur**(*nom_film*, *nom_acteur*)

Soit la requête suivante :

```
SELECT nom_directeur, nom_acteur
FROM Directeur Join Acteur
WHERE Directeur.nom_film = Acteur.nom_film
```

Pour évaluer cette requête, il faut calculer la jointure **Directeur** ⋈ **Acteur**. Pour l'exécution des jointures, donnez :

- Algorithme
- Complexité

Dans les deux cas on tiendra compte des mouvements entre mémoire centrale et disque et on évaluera le nombre d'entrées-sorties.

Exercice A : *Algorithme avec boucles imbriquées ;*

Solution :

Algorithme

*On suppose que les relations **Directeur** et **Acteur** sont stockées séquentiellement : elles ne possèdent aucun chemin d'accès privilégié (aucun index) (Figure 7.1). L'algorithme le plus simple pour évaluer la jointure **Directeur** ⋈ **Acteur** consiste à comparer chaque nuplet de la relation **Directeur** à tous les nuplets de la relation **Acteur**, puis à concaténer les nuplets pour lesquels **Directeur**.nom_film = **Acteur**.nom_film et enfin à ajouter le nouveau tuple au résultat. L'algorithme est esquissé dans le Tableau 7.1¹ :*

Pour évaluer le coût (le nombre des E/S) de cet algorithme il faut connaître les paramètres suivants : (a) la taille de chacune de deux relations et (b) la taille disponible en Mémoire Centrale pour réaliser cette opération.

*Supposons que les deux relations à joindre ont une taille supérieure à la taille de l'espace Mémoire Centrale disponible. On suppose un tampon en mémoire centrale de $b + 2$ pages. On affecte en Mémoire centrale b pages (tampon principal) pour lire des nuplets de la table **Directeur**, une page (tampon auxiliaire) pour lire des nuplets de la table **Acteur** et une page pour stocker le résultat de*

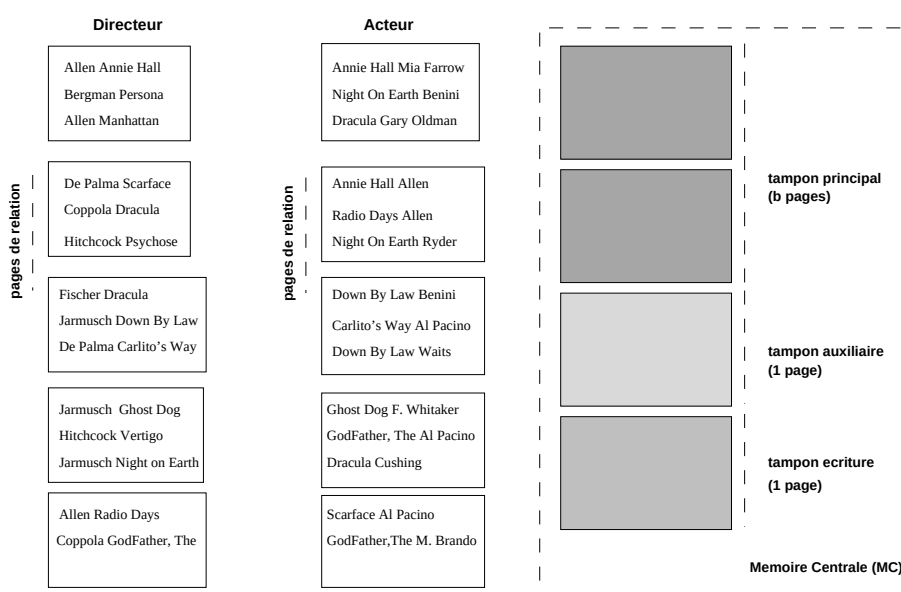
1. Pour l'algorithme on joint les relations R, S où l'attribut de jointure est a .

```

Résultat = ∅
tant que R n'est pas finie
  lire tuple  $t_R$  de R
  tant que S n'est pas finie
    lire tuple  $t_S$  de S
    si  $t_R.a = t_S.a$ , Résultat = Résultat  $\cup \{ t_R \bowtie t_S \}$ 
  ftq
ftq

```

TABLE 7.1 – Esquisse de l'algorithme des Boucles Imbriquées

FIGURE 7.1 – Relations **Directeur** et **Acteur** sur disque, et image de la Mémoire Centrale

la jointure (tampon écriture). On charge séquentiellement la relation **Directeur** par paquets de b pages dans le tampon principal (Figure 7.1).

Quand le tampon principal est plein, on charge la première page de la relation **Acteur** dans le tampon auxiliaire. Ensuite, on compare chaque nuplet du tampon principal à tous les nuplets du tampon auxiliaire. Les nuplets qui vérifient le critère de jointure sont joints. Le nuplet résultat est alors écrit dans le tampon écriture. Lorsque le tampon écriture est plein, son contenu est écrit sur disque.

Une fois que la jointure des b premières pages de **Directeur** est effectuée avec la première page de **Acteur**, on réitère le traitement en chargeant la seconde page de **Acteur** dans le tampon auxiliaire et ainsi de suite jusqu'à épuisement de la relation **Acteur**. A ce moment là, tous les nuplets des b premières pages de la relation **Directeur** chargées en MC, ont été comparés avec tous les nuplets de la relation **Acteur**. On réitère alors le traitement en chargeant les b pages suivantes de la relation **Directeur** dans le tampon principal, et ce jusqu'à épuisement de la relation **Directeur**.

Performances

La relation **Directeur** est lue une seule fois $(p_{\text{Directeur}} E/S)^2$ par morceaux de b pages. Pour chaque morceau, la relation **Acteur** est lue entièrement. Alors, la relation **Directeur** doit être découpée en

2. Ici p_R dénote le nombre de pages de la relation R , et n_R dénote le nombre de nuplets de la relation R .

$\lceil p_{\text{Directeur}}/b \rceil$ morceaux (où $\lceil A \rceil$ dénote la partie entière supérieure de A). Le nombre total de lectures générées par l'algorithme de jointure par boucles imbriquées est :

$$\text{Coût}_{E/S}(\text{Join_Boucles_Imbr}) = p_{\text{Directeur}} + \lceil p_{\text{Directeur}}/b \rceil \times p_{\text{Acteur}}$$

Le nombre d'écritures dépend de la taille du résultat. Compte tenu de la formule du coût d'E/S on a intérêt à charger la plus petite relation dans le tampon principal et à faire défiler la plus grande relation dans le tampon auxiliaire. Dans le cas où la plus petite relation tient en MC, alors le coût d'E/S devient $p_{\text{Directeur}} + p_{\text{Acteur}}$.

Le nombre de comparaisons en mémoire centrale est $n_{\text{Directeur}} \times n_{\text{Acteur}}$.

Exercice B : Algorithme avec tri-fusion ;

Solution :

La première étape consiste à trier chacune des relations **Directeur** et **Acteur** sur leur attribut de jointure (`nom_film`) (Figure 7.2). La deuxième étape (fusion) nécessite pour son exécution trois tampons du Mémoire Centrale, d'une page chacun. Les deux premiers tampons permettent de lire séquentiellement les relations **Directeur** et **Acteur** page par page et le troisième tampon est réservé pour écrire le résultat de la jointure.

Algorithme

L'algorithme consiste à positionner un pointeur courant $pt_{\text{Directeur}}$ (pt_{Acteur}) sur le premier nuplet de la relation **Directeur** (**Acteur**) et à comparer les attributs de jointure de ces deux nuplets. L'algorithme répète le test suivant :

- Si l'attribut **Directeur**.nom du nuplet pointé par $pt_{\text{Directeur}}$ est inférieur à l'attribut **Acteur**.nom du nuplet pointé par pt_{Acteur} , on incrémente $pt_{\text{Directeur}}$ afin qu'il pointe sur le nuplet suivant de **Directeur**.
- Si l'attribut **Directeur**.nom du nuplet pointé par $pt_{\text{Directeur}}$ est supérieur à l'attribut **Acteur**.nom du nuplet pointé par pt_{Acteur} , on incrémente pt_{Acteur} afin qu'il pointe sur le nuplet suivant de **Acteur**.
- Si les deux nuplets courants ont leurs attributs de jointure égaux, on génère un nuplet résultat et on incrémente l'un des deux pointeurs.

Lorsque l'un des pointeurs courants pointe sur le dernier nuplet d'une page, son incrémentation génère la lecture de la page suivante de la relation, dans le tampon associé à cette relation, et le pointeur est positionné sur le premier nuplet de cette nouvelle page.

Une variante de l'algorithme ci-dessus peut être utilisée pour les theta-jointures autres que l'équi-jointure.

Performances

Le nombre d' E/S de la jointure par tri-fusion comprend le coût éventuel du tri de la (ou des) relations à joindre³ plus le coût de lecture séquentielle des deux relations à joindre, plus le coût d'écriture sur disque du résultat (ce dernier est exclus de la formule ci-dessous)

$$\text{cout}(\text{Join_tri_fusion}) = \begin{cases} p_S + p_R, & \text{si } R, S \text{ sont déjà triées,} \\ 2p_R \times \log_b p_R + p_S + p_R, & \text{si } S \text{ est triée} \\ 2p_S \times \log_b p_S + p_S + p_R, & \text{si } R \text{ est triée} \\ 2p_R \times \log_b p_R + 2p_S \times \log_b p_S + p_S + p_R, & \text{si ni } R, \text{ ni } S \text{ ne sont triées} \end{cases}$$

Le coût mémoire centrale de fusion de deux listes triées de n_R et n_S nuplets dans le pire des cas est quadratique. Dans un cas non dégénéré il est linéaire en $n_R + n_S$. Noter aussi que dans le cas dégénéré, l'algorithme montré plus haut ne marche pas. Pourquoi ?

3. Le coût du tri d'une relation de p pages est de l'ordre de $2p \times \log_b p$ E/S b étant le nombre de pages du tampon en mémoire centrale disponible pour le tri.

	Directeur	Acteur
pages de relation - - - - -	Allen Annie Hall De Palma Carlito's Way Jarmusch Down By Law	Annie Hall Allen Annie Hall Mia Farrow Carlito's Way Al Pacino
	Fischer Dracula Coppola Dracula Jarmusch Ghost Dog	Down By Law Benini Down By Law Waits Dracula Cushing
	Coppola GodFather, The Allen Manhattan Jarmusch Night on Earth	Dracula Gary Oldman Ghost Dog F. Whitaker GodFather, The Al Pacino
	Bergman Persona Hitchcock Psychose Allen Radio Days	GodFather, The M. Brando Night On Earth Benini Night On Earth Ryder
	De Palma Scarface Hitchcock Vertigo	Radio Days Allen Scarface Al Pacino

FIGURE 7.2 – Relations **Directeur** et **Acteur** triées sur *nom_film*